DRIVER PULSE

System Design Document

Project	Driver Pulse — Rideshare Driver Shift Intelligence
Document Type	System Design Document (SDD)
Scope	Full-stack production architecture for sensor processing, signal fusion, and driver dashboard
Version	1.0 — Hackathon Release
Status	Prototype complete — Production roadmap defined
Prepared By	Team Popcorns

TABLE OF CONTENTS

1. Functional Requirements — *What the system must do*
2. Non-Functional Requirements — *Performance, reliability, and quality constraints*
3. Data Pipeline — *Ingestion, transformation, and output flow*
4. Component Architecture — *Module boundaries and responsibilities*
5. Edge Processing Design — *On-device computation and bandwidth strategy*
6. Cloud Scalability Plan — *Horizontal scaling and infrastructure roadmap*
7. Privacy Architecture — *Data minimisation, consent, and compliance*
8. Offline Mode Handling — *Disconnected operation and sync strategy*
9. Failure Recovery — *Resilience, fault tolerance, and alerting*

1 Functional Requirements

What the system must do

FR-1 Sensor Signal Ingestion

The system must continuously ingest two sensor streams per active trip: accelerometer readings (x, y, z axes, speed) and cabin audio intensity samples (dB level, duration). In the prototype these are ingested from structured CSV files. In production they are delivered as a real-time stream from a mobile SDK polling the device hardware.

Req ID	Requirement	Engine	Status
FR-1.1	Ingest accelerometer readings with timestamp, trip_id, x, y, z, speed_kmh	sensor_engine	Implemented
FR-1.2	Ingest audio intensity samples with db_level and duration_sec	sensor_engine	Implemented
FR-1.3	Validate all input schemas before processing; emit [WARN] on missing columns	all engines	Implemented
FR-1.4	Accept streaming sensor events at up to 50 Hz without data loss	edge SDK	Roadmap

FR-2 Motion Event Detection

The system must evaluate each accelerometer reading against seven independent checks and label each reading with an event type and severity.

Req ID	Check	Threshold	Status
FR-2.1	Harsh braking detection	Longitudinal deceleration > 2.5 m/s^2	Implemented
FR-2.2	Sudden acceleration detection	Longitudinal acceleration > 2.5 m/s^2	Implemented
FR-2.3	Sharp turn / swerve detection	Lateral magnitude $\sqrt{(x^2+y^2)}$ > 2.0 m/s^2	Implemented
FR-2.4	Speed bump detection	Z-axis drop < 7.5 m/s^2 at speed > 15 km/h	Implemented
FR-2.5	Speeding detection	GPS speed > 60 km/h	Implemented
FR-2.6	Jerk detection	Rate of change of total magnitude > 3.0 m/s^3	Implemented
FR-2.7	Overall magnitude check	$\sqrt{(x^2+y^2+\lvert z-9.8\rvert^2)}$ > 3.5 m/s^2	Implemented

FR-3 Audio Event Detection

Req ID	Check	Threshold	Status
FR-3.1	Argument-level noise detection	dB > 90 and duration > 3 seconds	Implemented
FR-3.2	Very loud cabin noise detection	dB > 80 and duration > 3 seconds	Implemented
FR-3.3	Sustained elevated noise detection	dB above baseline for > 60 continuous seconds	Implemented
FR-3.4	Mechanical noise exclusion (door slams)	Spike duration < 3 seconds → excluded from conflict scoring	Implemented
FR-3.5	Road noise suppression	Audio spike within 5s of speed bump event → suppressed	Implemented

FR-4 Stress Scoring and Explainability

Req ID	Requirement	Status
FR-4.1	Compute combined stress score per event: (motion × 0.65) + (audio × 0.35)	Implemented
FR-4.2	Apply 1.20× co-occurrence boost when both motion and audio signals fire simultaneously	Implemented
FR-4.3	Assign per-event severity band (used by _severity() in sensor_engine.py): NONE (<0.30), LOW (0.30–0.49), MEDIUM (0.50–0.74), HIGH (≥0.75). Note: the shift-level dashboard stress badge uses a separate STRESS_BADGE scale in merge_engine.py (NONE <0.20, LOW 0.20–0.39, MODERATE 0.40–0.64, HIGH ≥0.65) — these two scales are distinct.	Implemented
FR-4.4	Enforce 15-second cooldown between consecutive flagged moments per trip (regardless of event type)	Implemented
FR-4.5	Generate plain-English explanation for every flagged event	Implemented
FR-4.6	Suppress events within 20 seconds of trip start or end (trip boundary margin)	Implemented
FR-4.7	Suppress all motion events when vehicle speed < 5 km/h (stationary suppression)	Implemented
FR-4.8	Cap outlier sensor values at 3.5σ above mean to prevent hardware spikes	Implemented

FR-5 Earnings Intelligence

Req ID	Requirement	Status
FR-5.1	Compute per-trip earnings velocity: fare / (duration_min / 60) in local currency/hr	Implemented
FR-5.2	Rate each trip: excellent (≥₹400/hr), good (≥₹280/hr), average (≥₹180/hr), below_average (<₹180/hr)	Implemented
FR-5.3	Compute running cumulative velocity over shift with phase classification (early/mid/peak/late)	Implemented
FR-5.4	Compute needed_velocity to reach goal from current earnings position	Implemented
FR-5.5	Produce forecast status: achieved / on_track / at_risk / behind / dead_zone	Implemented
FR-5.6	Apply surge dampening when current velocity exceeds personal baseline by 2×	Implemented
FR-5.7	Detect dead zone: effective velocity < 50% of area median with > 1 hour remaining	Implemented
FR-5.8	Generate shift-phase-aware motivational messages citing specific velocity gaps	Implemented

FR-6 Signal Fusion and Trip Quality

Req ID	Requirement	Status
FR-6.1	Left-join sensor summaries with earnings summaries on trip_id	Implemented
FR-6.2	Assign sentinel stress_score = -1.0 for trips with no sensor match (not 0.0)	Implemented
FR-6.3	Compute trip quality from two-dimensional stress + earnings rating matrix	Implemented
FR-6.4	Produce driver shift summary: total earned, trip quality distribution, best route, goal forecast	Implemented
FR-6.5	Handle drivers with no goal row gracefully using None-safe accessor	Implemented

FR-7 Dashboard and Simulation

Req ID	Requirement	Status
FR-7.1	Render mobile-first driver dashboard (max-width 480px) showing all shift insights	Implemented
FR-7.2	Display earnings velocity chart, goal progress, trip quality breakdown, stress event timeline	Implemented

Req ID	Requirement	Status
FR-7.3	Simulate real-time sensor streaming via dataset replay with configurable injection interval	Implemented
FR-7.4	Cache processed data for the session lifetime (TTL=None) to prevent redundant pipeline re-execution on tab switches or chart interactions	Implemented
FR-7.5	Support multi-driver login via driver selection; isolate per-driver data views	Implemented

2 Non-Functional Requirements

Performance, reliability, and quality constraints

NFR-1 Performance

ID	Requirement	Target	Measurement
NFR-1.1	End-to-end latency (sensor to dashboard)	< 6 seconds	P95 measured over 1,000 trips
NFR-1.2	Engine processing time per trip update	< 200 ms	Benchmarked on mid-range mobile CPU
NFR-1.3	Dashboard chart render time	< 2 seconds	Measured on Streamlit refresh
NFR-1.4	Sensor ingestion throughput (production target)	50,000 events/sec fleet-wide	Load tested via Kafka consumer lag
NFR-1.5	Stream transport latency (edge to cloud)	< 500 ms P95	Measured at Kafka ingestion boundary

NFR-2 Reliability and Availability

ID	Requirement	Target
NFR-2.1	Driver dashboard API availability	99.9% uptime (< 8.7 hours downtime/year)
NFR-2.2	Pipeline crash tolerance	No crash on missing columns; warn and degrade gracefully
NFR-2.3	Output completeness	Every driver in input produces at least one shift summary row
NFR-2.4	Sentinel integrity	Trips with missing sensor data receive stress_score = -1.0, not 0.0
NFR-2.5	Trip data durability	All processed outputs persisted before acknowledgement

NFR-3 Scalability

ID	Requirement	Target
NFR-3.1	Concurrent active drivers	10,000+ simultaneous trips without throughput degradation

ID	Requirement	Target
NFR-3.2	Horizontal scale-out	Processing workers scale linearly; no shared mutable state across workers
NFR-3.3	Kafka partition key	driver_id ensures one driver's events never split across workers
NFR-3.4	Fleet storage growth	90-day rolling retention; older data archived to cold storage

NFR-4 Maintainability and Testability

ID	Requirement	Target
NFR-4.1	Engine purity	All three engines are pure functions: DataFrame in, DataFrame out, no side effects
NFR-4.2	Schema validation coverage	validate_schema() called at every engine entry point
NFR-4.3	Test isolation	Each engine independently unit-testable with synthetic DataFrames
NFR-4.4	Threshold configurability	All detection thresholds defined as named constants; no magic numbers in logic
NFR-4.5	Explainability coverage	Every flagged event row includes a human-readable description field

NFR-5 Security

ID	Requirement	Target
NFR-5.1	Audio data minimisation	Only dB intensity level stored; raw audio waveforms never captured
NFR-5.2	On-device processing boundary	Raw sensor values processed on-device; only feature vectors transmitted
NFR-5.3	Data in transit	All API communications over TLS 1.3 minimum
NFR-5.4	Driver data isolation	All data access scoped to authenticated driver_id; no cross-driver reads
NFR-5.5	PII minimisation	No passenger identity data stored; trip_id used as anonymous reference

3 Data Pipeline

Ingestion, transformation, and output flow

Pipeline Overview

The Driver Pulse data pipeline is a three-stage transformation chain. Each stage is stateless, deterministic, and independently testable. Data flows forward only; no stage reads back from a downstream stage.

Pipeline Architecture Principle

Stage 1 (Sensor Engine): Raw signals → Labelled events + stress scores
Stage 2 (Earnings Engine): Trip records + Goals → Velocity, forecasts, messages
Stage 3 (Merge Engine): Safety outputs + Earnings outputs → Quality + Shift summary

Stages 1 and 2 are PARALLEL — they have no dependency on each other.
Stage 3 (Merge) depends on the outputs of both Stage 1 and Stage 2.

Stage 1 — Sensor Pipeline

Inputs

Input File	Key Columns	Validation
accelerometer_data.csv	trip_id, timestamp, x, y, z, speed_kmh	validate_schema() — 7 required columns
audio_intensity_data.csv	trip_id, timestamp, db_level, duration_sec	validate_schema() — 4 required columns
trips.csv	trip_id, driver_id, start_time, end_time	Used for trip boundary margin calculation

Transformations

- run_accel_checks(df): Apply 7 independent threshold checks; annotate each row with event type, binary flag, and individual event score
- run_audio_checks(df): Apply 4 audio checks; filter door slams (< 3s duration); annotate with audio event type
- detect_flagged_moments(accel, audio, trips): Fuse both annotated streams; apply stationary suppression, trip margin filter, z-score cap, cooldown; compute combined stress score and co-occurrence boost
- build_trip_summaries_sensor(trips, flagged, accel, audio): Aggregate per-trip: motion count, audio count, peak stress, worst event type

- build_explainability_log(flagged, accel_df): Generate structured explainability log with signal type, raw value, threshold, event label, and plain-English description for each flagged event row

Outputs

Output File	Rows	Key Columns
flagged_moments.csv	One row per flagged event	trip_id, driver_id, timestamp, accel_type, audio_type, combined_score, severity, description
trip_summaries_sensor.csv	One row per trip	trip_id, stress_score, flagged_count, worst_event_type, motion_events_count, audio_events_count

Stage 2 — Earnings Pipeline

Inputs

Input File	Key Columns	Validation
trips.csv	trip_id, driver_id, fare, duration_min, pickup_location, dropoff_location	validate_schema() — 6 required columns
earnings_velocity_log.csv	driver_id, log_id, timestamp, elapsed_hours, cumulative_earnings, current_velocity	validate_schema() — 6 required columns
driver_goals.csv	driver_id, target_earnings, target_hours, shift_start_time, shift_end_time, current_hours	validate_schema() — 6 required columns

Transformations

- build_trip_earnings(trips): Compute earnings_velocity = fare / (duration_min / 60); assign earnings_rating; build route label
- build_enriched_velocity(velocity_log, goals): For each log entry, compute shift phase, needed velocity, forecast status, motivational message
- build_enriched_goals(goals, velocity_log, trips): Compute goal snapshot; apply surge dampening (2× baseline threshold); detect dead zone; project shift-end earnings

Outputs

Output File	Rows	Key Columns
trip_earnings.csv	One row per trip	trip_id, driver_id, fare, earnings_velocity, earnings_rating, route

Output File	Rows	Key Columns
enriched_velocity.csv	One row per velocity log entry	driver_id, timestamp, recalc_velocity, needed_velocity, shift_phase, forecast_status, message
enriched_goals.csv	One row per driver	driver_id, current_earnings, projected_earnings, effective_velocity, forecast, dead_zone, surge_adjusted, message

Stage 3 — Merge Pipeline

Inputs

trip_earnings.csv (from Stage 2), trip_summaries_sensor.csv (from Stage 1), enriched_goals.csv (from Stage 2), flagged_moments.csv (from Stage 1).

Join Strategy

A left join on trip_id preserves all trips in the earnings record. Trips with no sensor match (left join miss) receive a sentinel stress_score of -1.0. This sentinel is explicitly handled in _trip_quality() to return 'average' rather than falsely scoring the trip as calm (0.0).

```
merged = trip_earnings_df.merge(

sensor_summaries_df[['trip_id','stress_score','flagged_count','worst_event_type']],
    on='trip_id', how='left'
)
merged['stress_score'] = merged['stress_score'].fillna(-1.0)  # sentinel
```

Outputs

Output File	Rows	Key Columns
trip_summaries.csv	One row per trip	trip_id, driver_id, fare, earnings_velocity, earnings_rating, stress_score, flagged_count, trip_quality
driver_shift_summary.csv	One row per driver	driver_id, total_earned, avg_velocity, peak_stress_score, stress_badge, trip quality counts, best_route, forecast, goal_message, total_flags

4 Component Architecture

Module boundaries and responsibilities

Module Inventory

Module	File	Responsibility	Inputs	Outputs
Sensor Engine	sensor_engine.py	Motion + audio signal processing, stress scoring	accelerometer, audio, trips CSVs	flagged_moments, trip_summaries_sensor
Earnings Engine	earnings_engine.py	Velocity, forecasting, goal tracking	trips, velocity_log, goals CSVs	trip_earnings, enriched_velocity, enriched_goals
Merge Engine	merge_engine.py	Signal fusion, trip quality, shift summary	Stage 1 + Stage 2 outputs	trip_summaries, driver_shift_summary
Unified App	app_realtime_demo.py (sole entry point)	Mobile-first Streamlit dashboard (Home, Trips, Safety, Charts, Export tabs) + Live Sim tab for real-time streaming demo. app.py has been removed — this is the only app.	All output CSVs + realtime_trip_simulation.csv	None (read-only display)

sensor_engine.py — Interface

Function	Signature	Returns
run_accel_checks	run_accel_checks(df: DataFrame) → DataFrame	Annotated accelerometer readings with event labels and scores
run_audio_checks	run_audio_checks(df: DataFrame) → DataFrame	Annotated audio readings with event labels, door slam filter applied
detect_flagged_moments	detect_flagged_moments(accel, audio, trips) → DataFrame	One row per flagged event with combined stress score and severity
build_trip_summaries_sensor	build_trip_summaries_sensor(trips, flagged, accel, audio) → DataFrame	Aggregated per-trip sensor metrics

Function	Signature	Returns
build_explainability_log	build_explainability_log(flagged_df, accel_df) → DataFrame	Plain-English description added to each flagged event row

earnings_engine.py — Interface

Function	Signature	Returns
build_trip_earnings	build_trip_earnings(trips: DataFrame) → DataFrame	Per-trip earnings velocity, rating, and route label
build_enriched_velocity	build_enriched_velocity(velocity_df, goals_df) → DataFrame	Time-series velocity log with phase, forecast, and message
build_enriched_goals	build_enriched_goals(goals_df, velocity_df, trips_df) → DataFrame	Goal snapshot with projection, surge flag, and dead zone flag

merge_engine.py — Interface

Function	Signature	Returns
build_trip_summaries	build_trip_summaries(trip_earnings_df, sensor_summaries_df) → DataFrame	Master trip table: earnings + sensor merged, trip_quality computed
build_driver_shift_summary	build_driver_shift_summary(trip_summaries_df, enriched_goals_df, flagged_df) → DataFrame	One row per driver: full shift KPIs, stress badge, forecast, best route

Dependency Graph

↳ Data Flow Dependency Summary

```
CSV inputs (trips, accel, audio, goals, velocity_log)
   │
   ├─── sensor_engine.py ──────────────┐
   │      flagged_moments.csv          │
   │      trip_summaries_sensor.csv    │
   │                                   │
   └─── earnings_engine.py             │
        trip_earnings.csv              │
        enriched_velocity.csv          │
        enriched_goals.csv  ──────────▼
                          merge_engine.py
                             trip_summaries.csv
                             driver_shift_summary.csv
                                  │
                     ┌────────────┴────────────┐
                     app_realtime_demo.py (unified)
```

5 Edge Processing Design

On-device computation and bandwidth strategy

Motivation

Rideshare drivers carry Android or iOS devices with intermittent cellular connectivity, limited battery, and in-cabin microphones subject to privacy regulation. Moving as much processing as possible to the device (the edge) reduces network bandwidth, protects privacy by ensuring raw signals never leave the device, and allows the system to continue operating when connectivity is lost.

On-Device vs. Cloud Processing Boundary

Processing Stage	Location	Rationale
Raw accelerometer sampling (x, y, z, speed)	On-device	Raw signal never transmitted; processed locally
Raw audio dB sampling	On-device	Audio intensity only; no waveform ever captured or stored
Z-score outlier capping (ZSCORE_CAP = 3.5σ)	On-device	Eliminates hardware spike artifacts before transmission
Stationary suppression (< 5 km/h)	On-device	Filters majority of idle readings; reduces event volume by ~40%
Trip boundary margin filter (±20s)	On-device	Eliminates pickup/dropoff noise events before network send
Door slam classification (< 3s audio spikes)	On-device	Mechanical noise never enters cloud pipeline
Event type labelling (harsh brake, sharp turn, etc.)	On-device	Labels transmitted; raw values suppressed
Combined stress score computation	Cloud (or device)	Fusion step may move to device in Phase 2 for offline support
Earnings velocity calculation	Cloud	Requires trip fare data from platform API
Goal forecasting and shift summaries	Cloud	Aggregated over full shift; low latency not required

Sensor Sampling Strategy

The accelerometer polling rate is configurable and directly trades detection accuracy against battery consumption. The system is designed around a default of 5 Hz which is sufficient for all seven implemented checks.

Sampling Rate	Battery Impact	Detection Quality	Recommended For
50 Hz	High (~15% battery/hr)	Excellent — captures all jerk sub-events	Safety research and ML labelling
10 Hz	Medium (~8% battery/hr)	Good — all braking and turning events	Urban areas with frequent events
5 Hz (default)	Low (~4% battery/hr)	Good — sufficient for all 7 checks	Standard production deployment
1 Hz	Minimal (~1% battery/hr)	Limited — may miss short jerk events	Battery-critical sessions only

Memory Management on Device

The sensor engine is designed to operate on rolling time windows rather than accumulating full-trip buffers in memory. The two window constants that define the maximum in-memory lookback are:

```
STATIONARY_WIN  = 10   # seconds: sliding window for stationary detection
FLAG_COOLDOWN   = 15   # seconds: deduplication window per event type
END_TRIP_BUF    = 30   # seconds: end-of-trip suppression buffer
```

A trip of any length therefore requires only a fixed ~30-second sliding window in memory, not the full trip sensor log. This bounds peak device memory usage regardless of trip duration.

Bandwidth Optimisation

By running all threshold checks on-device and transmitting only labelled events (not raw readings), the upstream bandwidth requirement is reduced by approximately 95% compared to transmitting all sensor readings to the cloud.

Data Type	Raw Size (est.)	Transmitted Size (est.)	Reduction
Accelerometer readings @ 5 Hz, 1hr trip	~720 KB	~15 KB (event labels only)	98%
Audio dB samples @ 1 Hz, 1hr trip	~86 KB	~8 KB (event labels only)	91%
GPS/speed readings @ 1 Hz, 1hr trip	~200 KB	~200 KB (required for fare data)	0%

6 Cloud Scalability Plan

Horizontal scaling and infrastructure roadmap

Current Architecture (Prototype)

> ℹ️ **Prototype Scope**
>
> Single-process Python pipeline running on a local machine.
> All data stored in CSV files on the local filesystem.
> Streamlit cache (TTL = None, indefinite per session) prevents redundant reprocessing on tab switches.
> No external dependencies; fully self-contained; reproducible for hackathon evaluation.

Production Architecture Layers

Layer	Component	Technology	Scale Target
Mobile Edge	Sensor SDK + on-device preprocessor	Android/iOS SDK (Kotlin/Swift)	1 instance per active driver
Stream Ingestion	Event queue receiving edge payloads	Apache Kafka / Google Pub-Sub	50,000 events/sec fleet-wide
Stream Processing	Stateless engine workers consuming Kafka topics	Apache Flink or Spark Streaming	Horizontally scaled worker pool
Output Store	Time-series event storage and shift summaries	ClickHouse + PostgreSQL	90-day retention, daily rollups
API Layer	Driver-facing REST/ WebSocket API	Go or Node.js, behind load balancer	10,000 concurrent connections
Dashboard	Mobile-native or Streamlit driver interface	React Native / Streamlit	1 session per active driver
ML Inference	Trained stress event classifier (future)	TensorFlow Serving / Vertex AI	< 50ms inference per event

Horizontal Scaling Design

The pipeline is embarrassingly parallel at the driver level. Each driver's trip events can be processed by an independent worker with no cross-driver dependencies. The Kafka partition key is driver_id, which guarantees that all events from a single driver are always routed to the same worker and processed in order.

```
# Kafka topic: driver_pulse_sensor_events
```

```
# Partition key: driver_id
# Each partition consumer = 1 stateless engine worker
# Scale: add workers = add partitions, no rebalancing required
```

Because the sensor engine, earnings engine, and merge engine are all pure DataFrame transformations with no shared mutable state, adding more workers does not require distributed locking or coordination. Worker failures are handled by Kafka consumer group rebalancing.

Infrastructure Roadmap

Phase	Milestone	Deliverable	Target Scale
Phase 1 (Current)	Prototype	Python pipeline + Streamlit dashboard + CSV data	1 driver demo
Phase 2	Stream Ingestion	Kafka cluster + edge SDK + Flink processor deployed	100 concurrent drivers
Phase 3	Platform Integration	Real trip webhooks, fare data, surge multipliers from ride-hailing API	1,000 drivers
Phase 4	Database Layer	ClickHouse for events, PostgreSQL for summaries, BigQuery for analytics	10,000 drivers
Phase 5	ML Inference	Stress classifier + audio classification model + Vertex AI serving	10,000+ drivers
Phase 6	Mobile Native	React Native app replacing Streamlit; push notification service	Full production fleet

Database Partitioning Strategy

Data Store	Data Type	Partitioning Key	Retention
Kafka topic	Raw labelled sensor events	driver_id	7 days (replay window)
ClickHouse	Flagged moments (queryable)	driver_id + date partition	90 days
PostgreSQL	Shift summaries, goal records	driver_id primary key	2 years
InfluxDB / BigQuery	Earnings velocity time-series	driver_id + hour bucket	1 year, hourly rollup
Object storage (S3/GCS)	Raw trip CSVs, audit logs	date/driver_id path prefix	7 years (compliance)

7 Privacy Architecture

Data minimisation, consent, and compliance

Privacy Design Principles

🔒 Core Privacy Commitments

1. No raw audio ever captured, stored, or transmitted at any layer.
2. Audio dB intensity level is the only audio-derived signal in the system.
3. On-device processing boundary ensures raw sensor values are never sent to the cloud.
4. Passenger identity is never stored; trip_id is an opaque anonymous reference.
5. Driver data is strictly scoped to the authenticated driver_id; no cross-driver reads.

Audio Privacy Implementation

The audio privacy boundary is enforced architecturally, not just by policy. The mobile SDK captures only the decibel level of ambient sound in the cabin using the device microphone's intensity API, not the audio waveform. There is no audio buffer, no audio file, and no audio stream anywhere in the system.

What is NOT captured	What IS captured
Raw audio waveform	dB intensity level (a single number per sample)
Voice content or speech	Duration of the intensity sample in seconds
Speaker identity	Timestamp of the sample
Language or words spoken	Event label (argument / elevated / normal) after on-device classification
Audio recording file	Nothing — no audio file is ever created

Data Classification

Data Type	Classification	Storage Location	Access Scope
Flagged event labels + stress scores	Driver operational data	ClickHouse / local CSV	Authenticated driver only
Trip earnings and velocity	Driver financial data	PostgreSQL / local CSV	Authenticated driver only
Shift summaries and goal forecasts	Driver performance data	PostgreSQL / local CSV	Authenticated driver only
Audio dB intensity samples	Anonymised sensor data	Processed on-device; event label only in cloud	Authenticated driver only

Data Type	Classification	Storage Location	Access Scope
trip_id references	Anonymous operational identifier	All stores	Driver and platform system
driver_id	Pseudonymous identifier	All stores	Driver and platform system
Passenger data	NOT STORED	N/A	N/A — no passenger PII in system

Consent and Transparency

In production, the Driver Pulse feature requires explicit opt-in from the driver before sensor collection begins. The consent flow must disclose:

- That the device accelerometer will be sampled during active trips
- That cabin audio intensity (not audio content) will be measured during active trips
- That both signals are used to compute a driving safety score
- That the driver can view all flagged events and the explanation for each flag
- That data is retained for 90 days and can be deleted on request

Regulatory Compliance Considerations

Regulation	Relevance	Compliance Approach
GDPR (EU)	Driver is a data subject; stress score is derived personal data	Explicit consent; right to access; right to erasure; 90-day retention with deletion API
PDPB (India)	Indian drivers; biometric / health-adjacent data concerns	Opt-in consent; data localisation; no profiling without consent
CCPA (California)	Relevant if system deployed in California	Disclosure of data collected; opt-out of sale; access and deletion rights
Platform driver agreements	Ride-hailing platform terms govern sensor access	Legal review required before production deployment

8 Offline Mode Handling

Disconnected operation and sync strategy

Offline Design Philosophy

Driver Pulse is designed to be fully functional without a network connection during trip processing. A driver in a cellular dead zone, a tunnel, or a low-signal area should experience no degradation in safety event detection, stress scoring, or local dashboard updates. Cloud sync happens when connectivity is restored, not as a blocking dependency.

⚠️ **Prototype scope note —** The offline handling described in this section (write-local-first pattern, sync agent, conflict resolution, sync queue) is production design intent, not implemented in the current prototype. The prototype has no network layer. All outputs are written directly to the local outputs/ directory. These features are scoped to the post-hackathon roadmap and are documented here to demonstrate production-readiness of architectural thinking.

What Works Offline

Feature	Offline Capable	Notes
Accelerometer event detection	Yes	Runs entirely on-device; no network dependency
Audio intensity event detection	Yes	Runs entirely on-device; no network dependency
Stress score computation	Yes (on-device in production)	Currently on server; Phase 2 moves to device
Local trip output writing	Yes	All outputs written to local filesystem first
Dashboard display (last sync)	Yes	Shows last-synced data while offline
Real-time earnings velocity	Partial	Uses locally cached trip data; fare accuracy depends on platform sync
Goal forecast updates	Partial	Uses cached goal and trip data; may lag actual fare data
Cloud output sync	No	Queued for sync on reconnect; does not block trip processing

Local Write-First Pattern

The system follows a write-local-first pattern. All engine outputs are written to the local filesystem before any network operation is attempted. On reconnect, a sync agent reads all unsynced output files and pushes them to the cloud store in chronological order.

```
# Pseudocode: offline-safe processing loop
for each trip:
    outputs = run_pipeline(trip_sensor_data)
    write_local(outputs)          # always succeeds
    mark_unsynced(trip_id)        # queue for later
```

```
on_connectivity_restored():
    for trip_id in get_unsynced_queue():
        push_to_cloud(read_local(trip_id))
        mark_synced(trip_id)
```

Offline Data Handling Edge Cases

Scenario	System Behaviour
Phone loses connectivity mid-trip	Processing continues using local data; cloud sync queued; no data loss
Phone regains connectivity after multiple trips	All unsynced trips pushed to cloud in order; duplicate prevention via trip_id deduplication
Platform fare data not available offline	Trip processed with last-known fare; earnings velocity flagged as estimated; updated on sync
Driver goal not cached locally	Earnings forecast degrades to unknown status; stress scoring unaffected
Local storage full (< 10% free)	Oldest synced outputs purged from local store; unsynced outputs preserved
Device restarted mid-trip	Trip marked as incomplete; processed with available data; flagged in output as partial

Sync Conflict Resolution

If both a local offline write and a cloud-side update exist for the same trip_id, the system applies a last-write-wins strategy at the field level, with the local on-device measurement always taking precedence over cloud-derived estimates for sensor-based fields (stress score, flagged events). Cloud-derived fields (fare, surge multiplier) always take precedence over locally estimated values.

9 Failure Recovery

Resilience, fault tolerance, and alerting

Failure Taxonomy

Failure Class	Examples	Severity
Input schema failure	Missing required column in CSV input	Low — warns and degrades
Sensor hardware failure	Accelerometer returns constant zero, NaN values, or disconnects	Medium — outlier cap + suppression
Missing data failure	Trip has no sensor coverage (no accelerometer data for that trip_id)	Low — sentinel value handling
Driver goal not found	Driver in shift summary has no matching goal row	Low — None-safe accessor returns unknowns
Network connectivity loss	Stream transport to cloud fails during trip	Low — local write + queued sync
Worker process crash	Engine process OOM or unhandled exception	Medium — Kafka offset replay
Clock skew / timestamp gap	Phone clock incorrect; timestamps non-monotonic	Medium — elapsed-time calculations unaffected
Full storage failure	Local disk full; cannot write outputs	High — trip marked failed; alert triggered
Cloud database unavailable	ClickHouse or PostgreSQL unreachable	High — outputs queued locally; alert triggered

Schema Failure Handling

Every engine entry point calls validate_schema() before processing begins. Missing columns emit a structured warning log and the function returns False. The caller decides whether to abort or continue with degraded output.

```
def validate_schema(df: pd.DataFrame, required: list, context: str) -> bool:
    missing = [c for c in required if c not in df.columns]
    if missing:
        print(f'[WARN] {context} — missing columns: {missing}')
        return False
    return True
```

> ℹ️ **Degradation Contract**
>
> If schema validation fails for accelerometer_data:
> → Sensor engine returns empty flagged_moments.csv
> → Merge engine receives no sensor data; all trips get sentinel stress_score = -1.0
> → Trip quality defaults to 'average'; pipeline completes without crash
>
> If schema validation fails for driver_goals:
> → Earnings engine returns enriched_goals with forecast = 'unknown'
> → Shift summary produced with no goal fields; dashboard shows 'Goal data unavailable'

Sensor Hardware Failure Handling

Hardware Failure Mode	Detection Method	Recovery Action
Constant zero readings (sensor frozen)	ZSCORE_CAP capping produces all-zero capped values; no events fire	Emit low-confidence flag; tag trip as sensor-degraded
NaN / null sensor values	pandas fillna(0) applied before threshold checks; NaN rows suppressed	Continue processing; null rows excluded from event count
Accelerometer disconnected mid-trip	Timestamp gap > 30s with no readings detected	Mark trip as partial; process available segment; flag in output
Audio intensity stuck at 0 dB	Zero-variance audio series detected over 5-minute window	Suppress audio signal for trip; sensor flag added to output
GPS speed unavailable	speed_kmh column missing or all-null	Stationary suppression bypassed; speeding check disabled for trip

Worker Process Recovery

In the production Kafka-based architecture, each engine worker is a stateless consumer. If a worker crashes, Kafka retains the unacknowledged offsets and rebalances them to a healthy worker. Because all engine operations are idempotent (same input always produces same output), reprocessing a batch that was partially processed does not produce duplicate or corrupted output. The output store uses trip_id as a upsert key to deduplicate on write.

```
# Kafka consumer pattern: at-least-once delivery
# Output store upsert: INSERT ... ON CONFLICT (trip_id) DO UPDATE SET ...
# Result: exactly-once semantics at the output layer
```

Alerting and Observability

Alert Condition	Trigger	Action
Schema validation failure rate	> 5% of trips in a 5-minute window have missing columns	PagerDuty alert to on-call engineer
Kafka consumer lag	> 10,000 unprocessed events per partition for > 60 seconds	Auto-scale worker pool; alert if lag persists > 5 minutes
Trip processing error rate	> 1% of trips fail pipeline with unhandled exception	Alert; replay failed trips from Kafka offset
Dashboard API latency	P95 > 3 seconds for > 2 minutes	Alert; investigate database query performance
Local disk usage on device	> 90% of allocated trip storage	Purge oldest synced outputs; notify driver app
Sync queue depth	> 50 unsynced trips per driver	Alert; prioritise sync on next connectivity window

Failure Recovery Summary

✅ **Resilience Design Summary**

The Driver Pulse pipeline is designed to never lose a trip's data due to a recoverable failure.

Key resilience properties:

- Schema failures degrade gracefully — pipeline always produces output files
- Sensor hardware failures produce partial output with explicit degradation flags
- Network failures queue outputs locally and sync on reconnect — no data loss
- Worker crashes are recovered by Kafka offset replay — idempotent processing
- Missing goal or driver data returns 'unknown' status — no crash or silent drop
- All failures are logged with structured [WARN] or [ERROR] context strings
- Dashboard always shows last-known good data while backend recovers